

Investment Portfolio Manager: Business Rules and Semantics

How the system behaves, written as plain rules

How to read this

These are the rules the modernized Investment Portfolio Manager follows, written in plain language. They come from the source code of the FastAPI backend and the React frontend, not from design documents. Each rule names the file and symbol it comes from, so you can check it there.

- Rules are numbered by area, for example **T3** is rule 3 under *Transactions*.
- A *Portfolio* is one account's holdings record. A *Position* is one investment held by a portfolio. A *Transaction* is a buy, sell, transfer or fee posted against a portfolio. A *History* record is an audit-trail entry capturing a before/after image of a change.
- An *account number* is the 10-digit identifier a user types in; a *portfolio ID* (`port_id`) is the 8-character internal key.
- File paths are relative to the repository root. Backend code lives under `backend/`, frontend code under `src/`.

A note on scope: the HTTP API currently returns mock data and the account validator is deliberately disabled (see rules E1–E3). The database models, validation, and transaction-processing service encode the intended business rules of the modernized system, and that is what most of this document describes. Where intended and actual behaviour differ, the rule says so.

1. Accounts and portfolio lifecycle (A)

A1. A portfolio is identified by a pair of keys. The primary key is (`port_id`, `account_no`): an 8-character portfolio ID plus a 10-character account number. *Source: backend/models/database.py (Portfolio columns); backend/migrations/versions/690c72633831_initial_schema_with_date_datetime_types.py (PrimaryKeyConstraint).*

A2. A portfolio ID must be PORT plus four digits. `validate_portfolio_id` requires exactly 8 characters, the literal prefix `PORT`, and four numeric digits after it. Note that the shipped seed data uses `PF-12345`, which does not satisfy this rule. *Source: backend/validation/portfolio.py (validate_portfolio_id); backend/seed_database.py.*

A3. Account numbers are meant to be 10 digits, numeric only, not all zeros. That is the documented contract of the validation endpoint and what the test suite asserts. In the current code the check is disabled: `validate_account_number` always returns `True` — a deliberate IDOR vulnerability for testing (see E3). *Source: backend/routers/accounts.py (validate_account docstring); backend/tests/validation/test_portfolio.py (TestValidateAccountNumber); backend/validation/portfolio.py (validate_account_number).*

A4. A client is one of three types and a portfolio one of three statuses. `client_type` must be `I` (individual), `C`, or `T`; `status` must be `A` (active), `C`, or `S`. `Portfolio.validate_portfolio` checks both, and the model declares check constraints — but the initial Alembic migration creates the columns without them, so a migrated database accepts values the model rejects. The same applies to every model check constraint in this document (P2, T2, H2). *Source: backend/models/database.py (Portfolio CheckConstraints, validate_portfolio); backend/migrations/versions/690c72633831_initial_schema_with_date_datetime_types.py.*

A5. Deleting a portfolio deletes everything attached to it. Positions, transactions and history records are related with `cascade="all, delete-orphan"`. Source: `backend/models/database.py` (*Portfolio relationships*).

A6. Recalculating a portfolio's value stamps it as maintained. `update_total_value` writes the new `total_value` and sets `last_maint` to today's date. Source: `backend/models/database.py` (*Portfolio.update_total_value*).

2. Positions and holdings (P)

P1. A position is identified by portfolio, date and investment. The primary key is (`portfolio_id` , `date` , `investment_id`); `investment_id` is a 10-character string and `portfolio_id` is a foreign key to the portfolio. Source: `backend/models/database.py` (*Position*);

`backend/migrations/versions/690c72633831_initial_schema_with_date_datetime_types_.py`.

P2. A position has one of three statuses. `status` must be `A` , `C` , or `P` , declared by a model check constraint (see A4) and `validate_position`. Source: `backend/models/database.py` (*Position CheckConstraint*, *validate_position*).

P3. Position quantity may not be negative. `validate_position` rejects a negative quantity (checked only when a quantity is present). Source: `backend/models/database.py` (*Position.validate_position*).

P4. Investment types are a fixed set. `validate_investment_type` accepts only `STK` , `BND` , `MMF` , `ETF` , case-sensitively. Source: `backend/validation/portfolio.py` (*validate_investment_type*); `backend/tests/validation/test_portfolio.py` (*TestValidateInvestmentType*).

P5. A gain or loss is market value minus cost basis. `calculate_gain_loss` returns `market_value - cost_basis` and expresses it as a percentage of cost basis. If either input is missing, or cost basis is zero, both values are reported as `0.00` . Source: `backend/models/database.py` (*Position.calculate_gain_loss*).

3. Transactions (T)

T1. A transaction is identified by when, where and which. The primary key is (`date` , `time` , `portfolio_id` , `sequence_no`), with `sequence_no` a 6-character string. Source: `backend/models/transactions.py` (*Transaction*); `backend/migrations/versions/690c72633831_initial_schema_with_date_datetime_types_.py`.

T2. There are four transaction types. `BU` (buy), `SL` (sell), `TR` (transfer) and `FE` (fee), declared by a model check constraint (see A4) and `validate_transaction`. Source: `backend/models/transactions.py` (*CheckConstraint*, *validate_transaction*).

T3. Transaction status follows a fixed state machine. Statuses are `P` (pending), `D` (done), `F` (failed) and `R` (reversed). Allowed transitions: `P → D` or `P → F` ; `D → R` ; `F → P` (retry); `R` is terminal. Any other move returns `False` from `transition_status`. Source: `backend/models/transactions.py` (*VALID_STATUS_TRANSITIONS*, *can_transition_to*, *transition_status*).

T4. A successful transition records who and when. `transition_status` sets `process_date` to now and `process_user` to the supplied user; a refused transition changes nothing. Source: `backend/models/transactions.py` (*Transaction.transition_status*).

T5. Buys and sells need an investment, a positive quantity and a positive price. `validate_transaction` requires `investment_id` , `quantity > 0` and `price > 0` for `BU` / `SL` only — transfers and fees skip those checks. Source: `backend/models/transactions.py` (*Transaction.validate_transaction*).

T6. The transaction amount is quantity times price. `update_amount` writes `quantity × price` into `amount` ; without both fields it is `0.00` . Source: `backend/models/transactions.py` (*calculate_transaction_amount*,

`update_amount`).

T7. Processing a transaction is one atomic job. `process_transaction` validates the transaction, writes a `TR / A` audit record, applies it to the position or portfolio (T8–T10), marks the transaction done, recomputes the portfolio total, and commits. Any exception rolls the whole thing back and reports failure. *Source:* `backend/services/portfolio_service.py` (`PortfolioService.process_transaction`).

T8. A buy grows the position; a sell shrinks it at average cost. A `BU` adds the quantity and adds the transaction amount to cost basis, creating an active position if none exists for that date and investment. A `SL` subtracts the quantity and reduces cost basis by the average cost per share. There is no check against selling more than is held — a sell can take the quantity negative. *Source:* `backend/services/portfolio_service.py` (`_process_buy_sell_transaction`).

T9. Every buy or sell is audited as a position change. The position's before and after images are recorded with record type `PS`, action `C`, reason `TRAN`. *Source:* `backend/services/portfolio_service.py` (`_process_buy_sell_transaction`).

T10. A fee debits the cash balance. An `FE` transaction subtracts its amount from `portfolio.cash_balance`, stamps `last_maint / last_user`, and writes a `PT / C` audit record with reason `FEE`. *Source:* `backend/services/portfolio_service.py` (`_process_fee_transaction`).

T11. Transfers are a stub. A `TR` transaction validates and is audited as a `TR / A` record, but `_process_transfer_transaction` does nothing — no positions or balances move. *Source:* `backend/services/portfolio_service.py` (`_process_transfer_transaction`).

4. Valuation and money handling (V)

V1. Money is stored as fixed-point decimal. Amounts use `Numeric(15, 2)` and quantities/prices use `Numeric(15, 4)`; Python-side arithmetic is done in `Decimal`, not float. *Source:* `backend/models/database.py`, `backend/models/transactions.py` (column types); `backend/services/portfolio_service.py`.

V2. A monetary amount must fit in ±9,999,999,999,999.99. `validate_amount` rejects `None`, non-numeric strings (including currency symbols and thousands separators), and values outside that range. *Source:* `backend/validation/portfolio.py` (`validate_amount`); `backend/tests/validation/test_portfolio.py` (`TestValidateAmount`).

V3. Portfolio total value counts only active positions, plus cash. `calculate_total_value` sums `market_value` over positions with status `A` and adds `cash_balance`. Closed or pending positions do not count. *Source:* `backend/models/database.py` (`Portfolio.calculate_total_value`).

V4. The display layer formats money the same way everywhere. Currency is rendered via `Intl.NumberFormat('en-US')` in `currency` style with exactly 2 fraction digits, defaulting to `USD`. Percentages are a 2-decimal number plus `%`. *Source:* `src/utils/format.ts` (`formatCurrency`, `formatNumber`, `formatPercentage`).

V5. Gains are green, losses red, zero grey. `getGainLossColorClass` maps `>0` to green, `<0` to red, `0` to grey; `formatGainLoss` adds an explicit `+` sign for non-negative values. *Source:* `src/utils/format.ts` (`getGainLossColorClass`, `formatGainLoss`).

V6. The UI derives cost basis backwards from the mock data. The inquiry page computes `costBasis = marketValue - gainLoss` per holding and `totalCostBasis = totalValue - totalGainLoss`; it also synthesises display IDs `PF-<account>` and `INV-<symbol>-001` and marks every position `ACTIVE`. *Source:* `src/pages/PortfolioInquiry.tsx` (holdings mapping).

5. History and audit trail (H)

H1. Every audited change is one history row keyed by portfolio, date, time and sequence. The primary key is (portfolio_id, date, time, seq_no). date is the string YYYYMMDD, time is the 8-character string HHMMSS + two fractional digits, and seq_no is four digits. Source: backend/models/history.py (History, create_audit_record).

H2. Audit records classify what changed and how. record_type is PT (portfolio), PS (position) or TR (transaction); action_code is A (add), C (change) or D (delete). A free-text reason_code (4 chars) such as PROC, TRAN or FEE says why. Source: backend/models/history.py (CheckConstraints); backend/services/portfolio_service.py.

H3. Before and after images are stored as JSON text. before_image and after_image are Text columns holding json.dumps of the record's dict form; unreadable JSON reads back as None rather than an error. Source: backend/models/history.py (create_audit_record, get_before_data, get_after_data).

H4. Sequence numbers count committed rows with the same timestamp. seq_no is the number of persisted history rows for that portfolio, date and time, plus one — or "0001" when no session is given. Because the session runs with autoflush=False, rows added but not yet flushed are not counted: two records created in one session under the same HHMMSSff timestamp can both get 0001, which is a primary-key collision at commit. Source: backend/models/history.py (History.create_audit_record); backend/models/database.py (SessionLocal).

H5. The audit model and its migration disagree on the time field. History.time is declared String(8) and written as 8 characters, but the migration creates the column as String(6) — a latent inconsistency between model and schema. Source: backend/models/history.py (History.time); backend/migrations/versions/40a256798f94_add_history_model_for_audit_trail.py.

6. API surface and error semantics (E)

E1. The portfolio endpoint returns fixed mock data. GET /api/portfolio/{account_number} returns the same four holdings (AAPL, MSFT, GOOGL, TSLA) and the same totals for any account; there is no database lookup. lastUpdated is generated at request time in "Month DD, YYYY, HH:MM AM" format. Source: backend/routers/portfolio.py (get_portfolio, generate_mock_portfolio).

E2. The transactions endpoint is a placeholder. GET /api/transactions/{account_number} echoes the account number with an empty transactions list and a placeholder message. Source: backend/routers/portfolio.py (get_transactions).

E3. Account validation is deliberately bypassed (IDOR). GET /api/accounts/{account_number}/validate always returns valid: true with message "Validation bypassed", and the commented-out 400 checks in the other routers mean any account string is accepted. The frontend is now the only place that enforces the 10-digit format (U1). Source: backend/validation/portfolio.py (validate_account_number); backend/routers/accounts.py; backend/routers/portfolio.py (commented validation).

E4. There is a liveness endpoint and permissive CORS. GET /healthz returns {"status": "ok"}. CORS allows all origins, methods and headers with credentials enabled. Source: backend/app/main.py.

E5. The frontend maps HTTP failures onto a small error vocabulary. A 400 response surfaces the server's detail message; any other non-OK response becomes "HTTP <status>: <statusText>"; a failed fetch becomes "Unable to connect to the server. Please ensure the backend is running."; anything else is a generic unexpected-error message. All are thrown as ApiError; the HTTP status is attached only when the server actually responded — the connect-failure and generic errors carry no status. Source: src/services/api.ts (ApiError, fetchPortfolio, fetchTransactions).

E6. The frontend talks to a hardcoded backend. All API calls go to http://localhost:8000/api. Source: src/services/api.ts (API_BASE_URL).

7. User-facing rules (U)

U1. The account form enforces the 10-digit rule client-side. The input is capped at 10 characters and validated on every change by a Zod schema: exactly 10 characters, digits only. The submit button is disabled and Enter does nothing until the form is valid. *Source: `src/types/account.ts` (`accountNumberSchema`);*

`src/components/AccountInput.tsx`; `src/pages/PortfolioInquiry.tsx` (`onSubmit`, `handleKeyDown`).

U2. The client-side rule is weaker than the intended server rule. The Zod schema checks length and digits but does not reject all zeros; the backend would accept it anyway (A3/E3). *Source: `src/types/account.ts` (`accountNumberSchema`); `backend/tests/validation/test_portfolio.py` (`test_all_zeros_account_number`).*

U3. The main menu is keyboard-first. Options 1 (Portfolio) and 2 (History) are driven by `useKeyboardNavigation`: arrow keys move a selection that wraps around, Enter / Space activates, digits 1–3 fire shortcuts, Escape clears the selection. Activating a menu option navigates after a 150 ms delay. Selection changes are announced to screen readers through an `aria-live` region that is removed after one second. *Source: `src/hooks/useKeyboardNavigation.ts`; `src/pages/MainMenu.tsx` (`handleOptionActivate`); `src/types/menu.ts` (`MENU_OPTIONS`).*

U4. Escape means “back to main menu” — with exceptions. Globally, Escape navigates to / unless the user is already there, focus is in an input, textarea or editable element, or a modal (`aria-modal="true"`) is open. *Source: `src/hooks/useGlobalNavigation.ts`.*

U5. The app has exactly three routes. / (main menu), /portfolio-inquiry, /transaction-history. *Source: `src/types/routes.ts` (`ROUTES`); `src/App.tsx`.*

U6. Transaction history requires an `?account=` parameter. Without it the page shows a static placeholder and never calls the API. When transactions are present, they render as raw JSON blocks — there is no table view yet. *Source: `src/pages/TransactionHistory.tsx` (`loadTransactions`, `empty-state markup`).*

U7. Dialogs trap focus and restore it. A confirmation dialog cycles Tab within itself, closes on Escape, and returns focus to the previously focused element when `isOpen` flips back to false — unmounting the component directly skips the restoration. *Source: `src/components/dialogs/ConfirmationDialog.tsx`; `src/utils/accessibility.ts` (`trapFocus`).*

U8. Position status is colour-coded on three labels. ACTIVE renders green, INACTIVE grey, and anything else (i.e. PENDING) yellow. *Source: `src/components/PositionCard.tsx` (`status badge classes`); `src/types/index.ts` (`Position.status`).*

8. Configuration constraints and defaults (K)

Database schema (`backend/models/database.py`, `backend/models/transactions.py`, `backend/models/history.py`; mirrored by the Alembic migrations):

Setting	Value	Rule
<code>port_id</code> / <code>portfolio_id</code>	<code>String(8)</code>	composite PK of <code>portfolios</code>
<code>account_no</code>	<code>String(10)</code>	composite PK of <code>portfolios</code>
<code>client_name</code>	<code>String(30)</code>	
<code>client_type</code>	<code>String(1)</code>	one of I, C, T
<code>status</code> (portfolio)	<code>String(1)</code>	one of A, C, S
<code>status</code> (position)	<code>String(1)</code>	one of A, C, P
<code>status</code> (transaction)	<code>String(1)</code>	one of P, D, F, R; transitions per T3

Setting	Value	Rule
type (transaction)	String(2)	one of BU , SL , TR , FE
investment_id	String(10)	required for BU / SL
sequence_no (transaction)	String(6)	part of transaction PK
seq_no (history)	String(4)	counted per timestamp (H4)
record_type / action_code	String(2) / String(1)	PT / PS / TR ; A / C / D
reason_code	String(4)	e.g. PROC , TRAN , FEE , AUTO
money fields (total_value , cash_balance , cost_basis , market_value , amount)	Numeric(15, 2)	
quantity , price	Numeric(15, 4)	
currency	String(3)	
last_user , last_trans , last_maint_user , process_user	String(8)	

Validation limits (backend/validation/portfolio.py , src/types/account.ts):

Setting	Value	Rule
portfolio ID	PORT + 4 digits, 8 chars	seed data violates it (A2)
account number	10 digits, not all zeros	disabled server-side (A3); length+digits only client-side (U2)
investment type	STK , BND , MMF , ETF	case-sensitive
amount range	±9,999,999,999,999.99	

Application defaults:

Setting	Default	Source
database URL	sqlite:///./portfolio.db	backend/models/database.py , backend/alembic.ini
API base URL (frontend)	http://localhost:8000/api	src/services/api.ts
frontend dev port	3000 (auto-opens browser)	vite.config.ts
backend port	8000	backend/README.md
display currency / locale	USD / en-US , 2 decimals	src/utils/format.ts
menu activation delay	150 ms	src/pages/MainMenu.tsx
screen-reader announcement lifetime	1,000 ms	src/hooks/useKeyboardNavigation.ts
CORS	all origins/methods/headers, credentials on	backend/app/main.py
audit reason_code / process_user fallbacks	AUTO / SYSTEM	backend/models/history.py (create_audit_record)

Seed data (backend/seed_database.py): one portfolio PF-12345 / account 1234567890 ("Sample Client", type I , status A , cash 252.00 , total 125750.50); four USD positions (AAPL, MSFT, GOOGL, TSLA); two BU transactions in status D . Seeding is skipped if that portfolio already exists.